

HRADA — Claude Code CLI Prompt (Complete Prototype Spec, EN)

How to use: open a terminal, create an empty project folder, run `claude`, then paste everything below starting from "Your Role" as your first message.

Cost note: this is a pure frontend prototype. It never calls any AI API at runtime, so running the app costs nothing. The only cost is Claude Code's own usage while it writes the code for you, billed against your subscription or API credits.

Your Role

You are a senior frontend engineer + product designer with strong motion-design skill, building a **production-quality clickable prototype** of **HRADA** to demo to investors and test with real HR staff.

Before writing any code, do the following in order:

1. Read this entire spec. Summarize back what you understood, and flag anything that seems contradictory or underspecified.
2. Propose a **short design plan before writing code** — palette, type, layout concept, and the principles you'll follow (see "Design Direction" below). Show it to me before you start building.
3. Ask up to 5 clarifying questions if anything is genuinely ambiguous.

Do not skip this even though it feels slow. This project is judged heavily on visual polish and motion quality, so planning first prevents a generic, templated result.

1. What HRADA Is — Full Context

An **AI Workforce Intelligence** platform for growing Thai SMEs (20–500 employees).

Tagline: *Put the Right Person in the Right Job and Grow the Right Skills.*

Core product loop:

RECRUIT → MATCH → DEVELOP → TRACK → new data feeds back into RECRUIT

Find the right person → Fit them to the right work → Grow the right skills → Track the growth

The problem HRADA solves: most organizations aren't short on talented people — they don't know what the people they already have are good at, or how to use them well. That information is scattered across resumes, HR databases, performance reviews, training records, project records, and manager assessments.

The core of the system is the Employee Skill Graph — instead of storing an employee as just "Marketing Executive," HRADA builds a full capability picture per person:

Employee

|— Skills (individual skills, each with a level)

|— Experience

|— Projects

|— Performance

|— Learning

|— Career Goal

|— Workload / Availability

Non-negotiable principle across the entire system: HRADA is a **Decision-Support System, not an AI that decides for humans**. Logic must always be: **Analyze → Recommend → Explain → Human Decides**, never **Analyze → Decide**. No feature may auto-reject or auto-decide anything about a person. Every recommendation must be explainable — this is about Fairness, Bias, Explainability, Trust, and Responsible AI.

Four user roles:

1. **HR / HR Manager** — Recruit, Employee Management, Skill Analysis, Learning, Tracking, Workforce Analytics
2. **Manager / Project Lead** — Create Projects, Find Teams, View team member skills, Request AI Recommendations
3. **Employee** — View own skills, own Skill Gap, own Learning Path, own Career Development

4. **CEO / Business Owner** — Workforce Health, Org-wide Skill Gaps, Talent overview, Workforce overview

Three Talent Classifications:

- **Core Expert** — deep expertise in one specific skill/domain
- **Bridge Member** — connects skills or people across multiple teams/domains
- **Developing Talent** — still has skill gaps but shows clear potential

What the prototype must make obvious within a few minutes:

Before HRADA — "We have talented people, but we don't really know what each of them is good at, or how best to use them." After HRADA — "We can see exactly what skills each person has, what role fits them, what they're missing, and how they should grow."

2. Tech Stack (Local Prototype via Claude Code)

Since this runs locally (not as a chat artifact), use a full real-project stack for maximum polish and smoothness:

- **Vite + React 18 + TypeScript** — scaffold with `npm create vite@latest`
- **Tailwind CSS** (latest) — arbitrary values are fine here, no artifact restrictions
- **Framer Motion** — all animation: page transitions, gestures, layout animation
- **Recharts** — line/bar/radar charts
- **Zustand** — global state (current role, selected employee, active filters)
- **React Router** — real multi-route navigation, not a single component with tab-switching
- **lucide-react** — icons
- **clsx + tailwind-merge** — conditional classNames
- **date-fns** — date handling

All data is **mock data in TypeScript files** (no backend, no real API calls). Persist interactive state (candidate decisions, checked learning steps, etc.) via Zustand's `persist` middleware into `localStorage` so demos survive page refresh — this is a local prototype, so `localStorage` is fine here (unlike a `claude.ai` artifact).

Suggested project structure:

`src/`

data/ all mock data (employees, skills, jobs, candidates, projects, learningCatalog)

lib/

scoring.ts all calculation functions (pure functions)

theme.ts design tokens

components/

ui/ reusable button, card, badge, tooltip

charts/

SkillConstellation/

pages/ the 8 screens

store/ zustand stores

App.tsx

3. Design Direction — Propose a Plan Before Coding

Work in two passes: **plan** → **check against the brief** → **build**.

3.1 Color System — Black + Blue + Sky

token	hex	where
ink	#070B12	main background, blue-black, never pure black
panel	#0F1826	cards, sidebar, panels, modals
panel-raised	#141F30	elevated card state (hover, active)

token	hex	where
line	#1C2C42	borders, dividers, chart gridlines
signal	#2563EB	primary blue — primary buttons, active states
sky	#38BDF8	bright sky blue — reserved exclusively for numbers/results the AI computed
haze	#8AA2C0	secondary text, labels, captions
text	#E6EEF9	primary text
warn	#F59E0B	risk / gap
critical	#F87171	critical severity

Color rule: `sky` is a signal meaning "this is what the AI computed." Never use it as decoration. Users should learn this association unconsciously within a few screens. Growth/positive values also use `sky` (not green) to keep the palette disciplined. If you use a gradient, it must run `signal` → `sky` only, and appear in at most 1–2 places total (e.g. the Dashboard hero).

3.2 Typography

- **Display/headings:** IBM Plex Sans Thai, weight 600
- **Body:** IBM Plex Sans Thai, weight 400
- **Numbers/data:** IBM Plex Mono with `font-variant-numeric: tabular-nums` — **every number in the system must use this font**
- Load via the `@fontsource` package, not a raw CDN link, for stability when running locally
- Type scale: 40 / 28 / 20 / 15 / 13 / 11 px only

3.3 Design Principles to Follow

- **Spend your boldness in one place:** the Skill Constellation on the Dashboard is the hero moment of the whole system. Everything else stays quiet and disciplined.
- **Avoid generic templated patterns:** identical border-radius/shadow on every card regardless of hierarchy, all-caps eyebrow labels above every heading, numbered

markers (01/02/03) that don't represent a real sequence, a "→" appended to every button.

- **Motion must be purposeful, not decorative.** Use it in exactly two situations: (a) responding to a user action (open, expand, confirm, filter), and (b) one orchestrated moment on the Dashboard's first load. Not fade-slide-up on every section by default.
- UI copy (buttons, empty states, errors) uses active voice describing exactly what will happen. Action names stay identical through the whole flow (a "Schedule Interview" button produces a "Interview scheduled" toast).

4. Signature Element — Skill Constellation (mandatory)

The Dashboard must feature a **Skill Constellation** as the system's visual centerpiece:

- Every employee is a node on a dark background, positioned in department clusters
- A thin sky-blue line connects two employees when they share a skill at level ≥ 3.0 — line thickness scales with the number of shared skills
- Node brightness/size scales with that person's total skill level
- Nodes for employees with workload $> 85\%$ pulse gently in **warn** orange (ambient, subtle)
- **Interaction:** hovering a node shows a tooltip with name + role; clicking a node navigates to that person's Employee Skill Profile with a continuous transition (use Framer Motion's **layoutId** so the node visually expands into the profile card)
- Build with SVG + Framer Motion; a lightweight approximated force-directed layout is fine (doesn't need to be physically accurate, just needs to look natural) — or pre-computed elegant positions if that reads as smoother
- On first load, nodes should fade/scale in group-by-group by department (this is the one orchestrated moment that's allowed)

Why: HRADA literally calls itself a Skill **Graph** — this element makes that word visually real and becomes the thing people remember after a demo.

5. Motion & Micro-interaction Spec

Detailed enough to guarantee the "smooth and polished" feel requested, while staying disciplined:

Situation	Behavior
Route change	Cross-fade + small vertical slide (8px), 200ms ease-out, via <code>AnimatePresence</code>
Opening/closing a panel (e.g. "Why this match?")	Slide in from the right with backdrop fade, medium-stiffness spring (no excessive bounce)
Re-ranking a list (e.g. candidate ranking)	Use Framer Motion's <code>layout</code> prop so cards visibly reorder, never jump
Button that triggers AI analysis (e.g. "Analyze team", "Search candidates")	600–900ms loading state with a specific message (e.g. "Analyzing the Skill Graph of 14 employees...") using a themed skeleton/progress indicator, not a generic spinner
KPI number changes	Animated counter (count up/down), never a hard jump
Charts (line/bar/radar)	Animate in (draw-in) on first mount only, not on every re-render
Hover on clickable buttons/cards	Background shifts to <code>panel-raised</code> + lifts 2px, 150ms — identical everywhere for consistency, not random shadows per card
Successful action confirmation (e.g. "Interview scheduled")	Toast slides in from top-right, auto-dismisses, check icon with a micro-bounce
Skill level bar/radar	Animate the fill only on first mount of the Employee Profile page
Empty states	Never static — a subtle breathing-effect icon/illustration plus copy telling the user what to do next

Every transition must respect `prefers-reduced-motion` — when set, strip all non-essential motion down to a short opacity fade only.

6. On-Screen Language

General text is in **Thai**. HR-specific terminology stays in **English**: Skill Gap, Match Score, Learning Path, Workforce Health, Core Expert, Bridge Member, Developing Talent, Promotion Readiness, Internal Mobility, Workload, Skill Coverage.

Correct: "พนักงาน 3 คนมี Promotion Readiness สูงกว่า 80%" Wrong: translating those terms into Thai neologisms that don't read naturally.

Writing style: buttons describe the real outcome, action names stay identical through a flow, empty states tell the user what to do next, error copy explains what happened and how to fix it — never a bare "an error occurred."

7. Skill Level Scale — Must Be Defined and Shown in the UI

Level	Name	Meaning
0–1.0	None	No experience with this skill
1.1–2.0	Aware	Understands the concept, needs guidance to perform it
2.1–3.0	Practicing	Handles routine work independently, still needs review
3.1–4.0	Proficient	Fully independent, can solve unfamiliar problems
4.1–4.7	Advanced	Mentors others, designs new ways of working
4.8–5.0	Expert	Sets the organizational standard, teaches others

A legend/tooltip explaining this table must be accessible from the Employee Skill Profile screen.

8. Role Permissions — Who Sees What

Data	HR	Manager	Employee
Own Skill Profile	✓	✓	✓
Others' Skill Profile	✓ org-wide	✓ own team only	✗
Others' Performance	✓	✓ own team only	✗
Others' Workload	✓	✓ own team only	✗
Salary Range	✓	✗	✗
Recruit / Candidates	✓	✗	✗
Team Matching	✓	✓	✗
Org-wide Workforce Insights	✓	own team only	✗

Switching role from the top-right switcher must actually change **routing and menus**, not just hide UI with CSS — if an Employee navigates directly to the Recruit URL, redirect them back with an explanation.

9. Mock Data — Use This Exact Dataset, Do Not Invent Randomly

9.1 Organization

Company "Thanawat Digital" ("ธนวัฒน์ ดิจิทัล"), Digital Services industry, 126 total employees (14 shown in the prototype). Departments: Marketing, Sales, Data, Product, Operations.

9.2 Skills — 18 total

- **Technical:** Digital Marketing, SEO, Data Analysis, SQL, AI Tools, Software Development
- **Analytical:** Financial Analysis, Market Research, UX Research

- **Creative:** Content Creation, Graphic Design, Copywriting
- **Leadership:** Leadership, Coaching, Project Management, Product Management
- **Communication:** Client Handling, Presentation

9.3 14 Employees (lock these values exactly)

#	Name	Title	Dept	Key skills (level)	Perf	Workload	Career Goal
1	Jenjira W.	Marketing Executive	Marketing	Digital Marketing 4.5, Content Creation 4.1, Client Handling 4.0, Data Analysis 3.2, Leadership 2.1	4.2	72%	Marketing Manager
2	Piya S.	Data Lead	Data	Data Analysis 4.8, SQL 4.7, AI Tools 3.9, Leadership 3.4	4.6	92%	Head of Data
3	Mark C.	Growth Marketer	Marketing	SEO 3.8, Digital Marketing 3.6, Data Analysis 3.5, Content Creation 3.4, Client	4.0	65%	Growth Lead

#	Name	Title	Dept	Key skills (level)	Perf	Workload	Career Goal
				Handling 3.3			
4	Araya T.	Content Specialist	Marketing	Content Creation 4.6, Copywriting 4.4, Graphic Design 3.5, SEO 2.4	4.3	70%	Creative Lead
5	Thanakorn M.	Junior Data Analyst	Data	SQL 2.9, Data Analysis 2.6, AI Tools 2.2	3.6	58%	Data Analyst
6	Suchada K.	HR Manager	Operations	Coaching 4.2, Leadership 4.0, Presentation 3.8, Project Management 3.2	4.4	76%	HR Director
7	Wichai P.	Sales Manager	Sales	Client Handling 4.6, Presentation 4.2, Leadership 3.9, Market Research 3.0	4.4	88%	Sales Director

#	Name	Title	Dept	Key skills (level)	Perf	Workload	Career Goal
8	Nicha A.	Sales Executive	Sales	Client Handling 3.3, Presentation 3.0, Content Creation 2.2	3.4	62%	Senior Sales Executive
9	Kitti R.	Product Manager	Product	Product Management 4.4, UX Research 3.6, Leadership 3.5, Data Analysis 3.3	4.3	80%	Head of Product
10	Pimchanok N.	UX Researcher	Product	UX Research 4.5, Presentation 3.4, Data Analysis 2.8	4.1	68%	Lead Researcher
11	Anucha W.	Software Developer	Product	Software Development 4.3, AI Tools 3.7, SQL 3.4	4.2	84%	Tech Lead

#	Name	Title	Dept	Key skills (level)	Perf	Workload	Career Goal
12	Methee J.	Ops Coordinator	Operations	Project Management 3.6, Client Handling 3.0, Presentation 2.8	3.5	66%	Ops Manager
13	Siriporn L.	Financial Analyst	Operations	Financial Analysis 4.4, Data Analysis 3.6, SQL 3.1	4.2	74%	Finance Manager
14	Phum T.	Marketing Intern	Marketing	AI Tools 2.6, Content Creation 2.4, Digital Marketing 2.0	3.8	45%	Marketing Executive

Fill in each person's remaining skills in the 1.0–3.0 range, sensibly matched to their role.

employmentType: #14 = Intern, #8 = Contract, everyone else = Full-time (also include a **Freelancer** option in the type enum for completeness, even if unused in this mock set).

Every employee needs: **skills[]** each with **evidence[]** (≥ 2 sources if level ≥ 3.0 , drawn from: **Project:** <name> / **Certification:** <name> / **Manager Review** <year> / **Assessment** <score> / **Peer Feedback**), **projects[]**, **learningHistory[]**, and a 6-month **skillHistory** (Mar–Aug 2026) for their top 3 skills.

9.4 Deliberately Planted Cases — Must Surface on Screen After Calculation

If any of these don't appear on screen, the formulas are wrong:

1. **Critical Skill Gap** → **AI Tools** — nobody in the org reaches 4.0
2. **At-Risk Skill** → **UX Research and Financial Analysis** — only 1 person meets the bar for each
3. **Workload Risk** → **Piya (92%, perf 4.6) and Wichai (88%, perf 4.4)**
4. **High Promotion Readiness** → **Jenjira, Mark, Kitti**
5. **Jenjira's Primary Skill Gap = Leadership** (currently 2.1, target role needs 4.0)
6. **Learning with no real outcome** → **Nicha** completed 2 Client Handling courses this year, but her skill only moved from 3.2 to 3.3 — must trigger a low-outcome warning
7. **Fast growth** → **Thanakorn and Phum** — 6-month skillHistory shows a steep upward trend, so they classify as Developing Talent

9.5 Open Jobs — 3 positions

1. **Senior Data Analyst** — required: Data Analysis 4.0, SQL 4.0, AI Tools 3.5 / preferred: Presentation 3.0 / min 4 years
2. **Content Marketing Lead** — required: Content Creation 4.0, Copywriting 3.5, SEO 3.5 / preferred: Leadership 3.0 / min 5 years
3. **Sales Executive** — required: Client Handling 3.5, Presentation 3.0 / preferred: Market Research 2.5 / min 2 years

Each job needs: title, department, description, responsibilities, requiredSkills, preferredSkills, minExperience, education, employmentType, location, salaryRange.

9.6 Candidates — 8 people

Invent 8 candidates with: education, yearsExperience, skills, certifications, projects, portfolio, assessmentScore (0–100).

Design them so these situations occur: (a) highest overall score but missing one critical skill (b) middling score but meets every required skill exactly (c) low experience but very high assessment score (d) two candidates with clearly low scores that **must not be auto-rejected**.

9.7 Projects — 3 total

1. **AI Marketing Campaign** — required: Digital Marketing 4.0, Data Analysis 3.5, Content Creation 3.5, AI Tools 4.0 / teamSize 4
2. **Customer Retention Program** — required: Data Analysis 3.5, Client Handling 4.0, Project Management 3.0 / teamSize 3
3. **Product Analytics Revamp** — required: SQL 4.0, UX Research 3.5, Software Development 3.5 / teamSize 3

9.8 Learning Catalog — 12 items

```
{ title, type  
(Course/Workshop/Project/Mentorship/On-the-job/Certification),  
targetSkill, difficulty(1-3), durationHours }
```

 Include at least 3 items with targetSkill = Leadership and 2 with targetSkill = AI Tools.

10. Calculations That Must Be Real — Never Hardcode a Number

Hard rule: every number on screen must come from a function in `src/lib/scoring.ts`. If you change one employee's skill level in mock data, every affected number on every screen must change too. Write these as pure functions with comments explaining the formula, and cover every one with a Vitest unit test.

10.1 `calcSkillGap(employee, targetRole)`

`gap = max(0, requiredLevel - currentLevel)`, sorted descending, with a `primaryGap` field.

10.2 `calcCandidateMatchScore(candidate, job)`

Weighted to 100: Required Skills 50% (`avg(min(candidateLevel/requiredLevel, 1))`) + Preferred Skills 15% (same formula) + Experience 15% (`min(years/minExperience, 1)`) + Project Relevance 10% (share of projects touching a required skill) + Assessment 10% (`assessmentScore/100`). Always return the total score **plus a per-category breakdown**. Flag `hasCriticalGap: true` if any required skill is more than 1.0 below the bar.

10.3 `calcTeamFit(employee, project)`

Skill Contribution 50% + Availability 20% (`((100-workload)/100)`) + Performance 15% (`(performance/5)`) + Project History 15%.

10.4 `selectTeam(project, employees, teamSize)` — greedy coverage

Never just take the top-N highest scores.

1. uncovered = all of the project's required skills
2. Loop until the team is full or uncovered is empty:
 - Count how many uncovered skills each candidate could satisfy (level $\geq$ required)
 - Pick whoever covers the most; break ties by higher teamFit
 - Remove the skills they cover from uncovered
3. Fill any remaining slots with highest teamFit among people with availability > 20%
4. If team size ≥ 4 , at least one member must be a Developing Talent —

if none qualify, swap out the lowest-fit member for the best-fit Developing Talent and explain why in the UI
5. If a selected member has workload > 90%, show a Workload Risk warning with a backup name
6. Return teamGaps = skills the team still doesn't meet, and this must link forward into creating a Job

Note: with this dataset, the AI Marketing Campaign project will always leave an AI Tools gap — make sure it links to the Recruit screen.

10.5 `classifyTalent(employee)`

Never hardcode "top 30% = Core Expert."

- **Core Expert** — has ≥ 1 skill at level ≥ 4.3 **and** ≥ 2 evidence sources for that skill **and** performance ≥ 4.0
- **Bridge Member** — has skills ≥ 3.2 spread across ≥ 3 different categories **or** ≥ 2 cross-department projects
- **Developing Talent** — still has a gap toward their target role **and** average growth from history $\geq +0.15/\text{month}$

If someone qualifies for multiple, pick whichever they score highest on, and always return a reason.

10.6 `calcPromotionReadiness(employee)`

$(\% \text{ of target role's required skills already met} \times 0.7) +$
 $(\text{performance}/5 \times 0.2) + (\min(\text{years in current role}/3, 1) \times 0.1)$

10.7 `calcWorkforceHealth(employees)`

Skill Coverage %, Critical Skill Gaps (fewer than 2 people meet the bar), High Potential (readiness $\geq 75\%$), At-Risk Skills (exactly 1 person meets the bar), Workload Risk (workload $> 85\%$ and performance ≥ 4.0), Internal Mobility.

10.8 `generateLearningPath(employee, catalog)`

From the primary gap $\rightarrow$ select matching catalog items by targetSkill, ordered easiest to hardest $\rightarrow$ **always finish with Real Project + Manager Assessment** (learning-in-the-flow-of-work principle; explain this in the UI).

10.9 `calcLearningOutcome(before, after)`

If delta $< 0.3 \rightarrow$ `lowOutcomeFlag: true` with copy like "this learning hasn't produced a measurable change yet — consider pairing it with real project work or mentorship."

10.10 `generateInsights(employees)`

Auto-generate at least 4 insight types: $\triangle$ Critical Skill Gap, $\triangle$ At-Risk Skill, $\triangle$ Workload Risk, $\uparrow$ Growth Opportunity — each must cite exactly what it was computed from.

10.11 Additional metrics (from the original product brief — do not drop these)

The source brief explicitly recommended these as the right KPI set for HRADA; add them to the Tracking screen alongside the above:

- `calcSkillCompletionRate(employee)` — % of assigned Learning Path steps marked complete
- `calcTimeToCompetency(employee)` — average number of months, using `skillHistory`, it took to close a `primaryGap` skill once it was identified (approximate using the first and last recorded history points that cross the required threshold)
- **Manager Satisfaction** — use the existing `performance` / manager-review evidence as the displayed proxy for this metric, labeled "Manager Satisfaction" in the UI

- **Employee Engagement in Development Plan** — proxy this as the % of assigned Learning Path steps *started* (not just completed) versus assigned; label it exactly as "Employee Engagement in Development Plan"

These four, together with Skill Coverage, Skill Gap Reduction (derivable from `calcSkillGap` over time), Internal Mobility Rate, and Performance Improvement After Learning (`calcLearningOutcome`), form the complete 6+1 KPI set from the original brief. Make sure all of them are visible somewhere in Screen 7 (Tracking), even if some are grouped under a "Learning Impact" subsection.

10.12 Conceptual mapping to the 4 named AI engines from the product spec

The original spec describes the AI as four logical engines. Your functions above implement them; make this mapping explicit in a code comment at the top of `scoring.ts` so the structure stays traceable:

- **Skill Intelligence Engine** → produces the Employee Skill Graph (skill level aggregation from evidence, §9.3's evidence-weighting logic)
- **Matching Engine** → `calcCandidateMatchScore`, `calcTeamFit`, `selectTeam`
- **Learning Engine** → `generateLearningPath`, `calcLearningOutcome`
- **Workforce Intelligence Engine** → `calcWorkforceHealth`, `generateInsights`, `calcPromotionReadiness`, `classifyTalent`

11. The 8 Screens — Do Not Drop Any Feature

Screen 1 — Role Switcher / Entry

Choose HR / Manager (pick which department they lead) / Employee (pick which of the 14 people they are). Menus change per §8. Switchable at any time from the top-right of every screen.

Also show, as a short illustrative strip on this entry screen (not a full page — a compact visual moment), the product's core loop: `RECRUIT` → `MATCH` → `DEVELOP` → `TRACK` with a small arrow back into `RECRUIT`. This orients first-time viewers before they pick a role and doubles as the "journey" narrative from the product brief.

Screen 2 — HR Workforce Dashboard

- KPI card row: Employees, Skill Coverage %, Critical Skill Gaps, High Potential, At-Risk Skills, Internal Mobility (40px numbers, mono font, **sky** color, animated counters)
- **Skill Constellation** (§4) as the hero
- Skill Coverage by department (bar chart, animates in)
- AI Insights panel from §10.10

Screen 3 — Employee Skill Profile

Select an employee (list or search) → name, title, department + all skills with levels (bar or radar chart, animated fill on mount)

- Clicking a skill reveals its evidence sources (smooth popover/expand)
- 6-month skill growth chart (line chart)
- Promotion Readiness + list of still-missing skills
- Skill Level Scale legend (§7), viewable on demand

Screen 4 — AI Recruit

Select a Job → ranked candidate list with Match Scores (cards animate reorder as scores/filters change)

- **"Why this match?"** button opens a slide-in panel with the §10.2 breakdown + strengths + Skill Gap
- **hasCriticalGap** warning shown distinctly, separate from the score
- HR decision buttons: Pass / Reject / Schedule Interview → confirmation toast on click. **Never auto-reject.**

Screen 5 — AI Team Matching

Manager selects a Project → system proposes a team per §10.4 (with an analysis loading state)

- Each member shows: Talent Type (color-coded badge), Fit %, skills they bring, workload, reasoning
- Adjustable team size (slider or +/-) → recomputes instantly with card enter/exit animation
- Shows teamGaps with an "Open a role" button linking to Recruit
- Workload Risk warning with a backup name

Screen 6 — Personalized Learning

Select an employee → Current Skills → Skill Gap → Target Role → Learning Path (§10.8)

- Learning Path steps are checkable (checkbox with a micro-animation; overall progress bar updates)
- Before/after Learning Outcome shown as an animated comparison bar (e.g. Leadership 2.1 → 3.4)
- Nicha's **lowOutcomeFlag** case must be visible here

Screen 7 — Tracking

Skill Growth (6-month line chart), Learning Progress (including the §10.11 additional KPIs), Performance (KPI / Goals / Project Outcome / Manager Assessment / Peer Feedback), Career Development (Career Goal, Promotion Readiness, Internal Mobility, Role Changes) Toggle between viewing an individual or a team (smooth transition)

Screen 8 — AI Workforce Insights

All insights, grouped by urgency (accordion or expandable cards)

- Expand to see exactly what was computed and which formula was used
- "Take action" button links to the relevant screen (Workload Risk → Team Matching, Skill Gap → Recruit or Learning)

Supplementary screen needed for a smooth demo (not one of the 8 core screens, but required)

- **404 / Not-authorized page** shown when an Employee tries to reach a screen they can't access — themed consistently, explains why access is denied.

12. The Rule That Must Never Be Broken — Explainability

Restated because it matters most: **Analyze → Recommend → Explain → Human Decides**. **Every single place** that shows a number or recommendation must be explainable on demand. No floating, unexplainable score anywhere. No button that lets the AI decide on a person's behalf.

13. Explicitly Out of Scope for This Prototype

✗ Real backend/database ✗ Real authentication (the role switcher is sufficient) ✗ Real LLM API calls ✗ Payroll ✗ Attendance ✗ Full LMS ✗ Full ERP/CRM ✗ A complete HR system

(For context only, not to build: the long-term vision is for HRADA to become an AI layer that connects to ERP, CRM, HRIS, ATS, LMS, Payroll, Attendance, and Performance systems — not to replace them from day one.)

14. Acceptance Checklist — Verify All of This Before Calling It Done

1. Every number comes from `src/lib/scoring.ts` — nothing hardcoded anywhere
 2. Every AI recommendation is explainable, and the explanation matches the actual breakdown
 3. `classifyTalent` never uses a "top 30%" rule
 4. `selectTeam` uses greedy coverage, never top-N by score
 5. All 7 planted cases in §9.4 visibly surface on screen
 6. Role permissions work per §8, including redirect-on-unauthorized-access
 7. Colors are limited to the tokens in §3.1; `sky` is used only for AI-computed results
 8. Every number uses the mono font with tabular-nums
 9. The Skill Constellation works, feels smooth, and is the system's one visual centerpiece
 10. Motion everywhere follows §5 — responds to user actions, nothing decorative or scattered
 11. `prefers-reduced-motion` is actually respected
 12. All 8 screens plus the not-authorized screen are reachable with no errors and no console warnings
 13. Language follows §6 — Thai UI text with English HR terminology
 14. `npm run build` passes with zero TypeScript errors
 15. Someone seeing this for the first time understands the §1 before/after summary within 3 minutes, unprompted
 16. The design avoids generic templated tells (identical cards with no hierarchy, eyebrow labels on every heading, "→" appended to every button, fake 01/02/03 numbering)
 17. The additional KPIs in §10.11 are visible in Screen 7, and the engine-mapping comment from §10.12 exists in `scoring.ts`
-

15. Recommended Build Sequence for Claude Code

Work in stages, committing periodically if git is available:

1. Scaffold the project + install dependencies + design tokens + fonts
2. Write all mock data (§9) + the entire scoring engine (§10) + unit tests
3. Main layout, routing, role store, role-aware sidebar/navbar
4. Screens 2 and 3, including the Skill Constellation
5. Screens 4 and 5, with the explanation panels and re-ranking animation
6. Screens 6, 7, 8
7. Run `npm run build`, verify against all 17 items in §14, and report back exactly what passed and what didn't, with reasons
8. Run `npm run dev` and tell me how to view the result

After each stage, give me a short summary of what was done and flag anything you decided on your own that I should know about.